HW2 | Step response from sampled frequency data

National Central University | EE8091

Introduction to Signal Integrity and Electromagnetic Compatibility

Result and scope

The Kaiser parameter **alpha = 1.269110238** produces a **40.000 ps** central 10%-90% input rise at a two-sided bandwidth F = 40 GHz. The reflected waveform is reconstructed from the supplied S11 and compared against a fresh, independent ngspice circuit simulation. All 9 saved numerical checks pass. Residual differences between the Kaiser and SPICE waveforms are quantified over a stated comparison interval.

This report covers Part I (five Kaiser steps and normalized rise time) and Part II (S-parameters, the 40 ps reflection, circuit validation, and alpha = 0 / 4 comparisons). All plots and metrics are regenerated by the accompanying Python program.

1. Transform and window conventions

The inverse transform uses frequency f in hertz:

```
h(t) = integral H(f) exp(+j 2 pi f t) df
hW(t) = IFT[H(f) W(f)] = h(t) * w(t)
sW(t) = integral[-infinity,t] hW(tau) d tau = h(t) * uW(t)
```

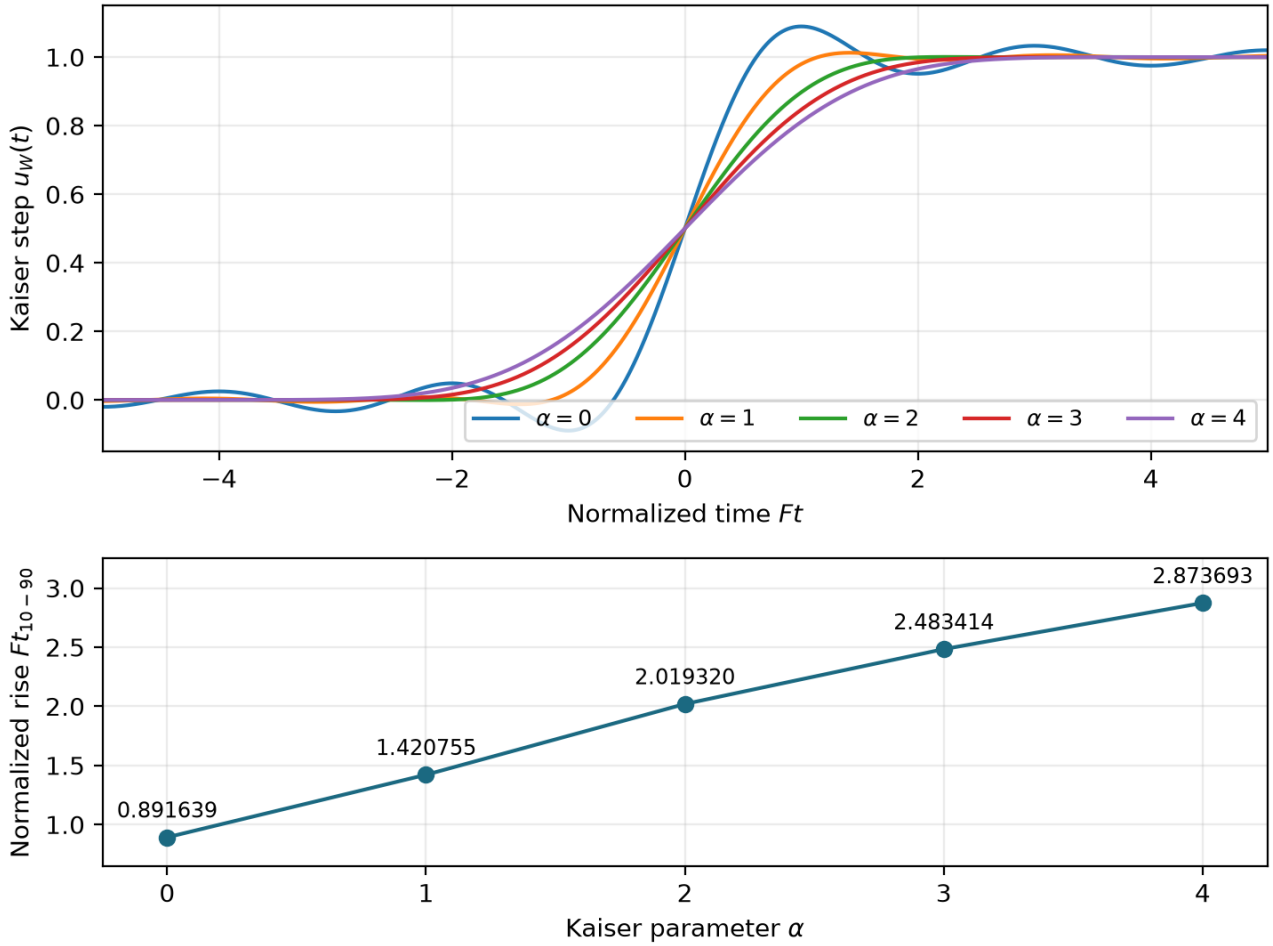
Here * denotes convolution. Integrating the windowed impulse response produces the response to the finite-rise input uW, rather than an ideal zero-rise step. The DC-centered Kaiser window is zero outside |f| <= F/2, and inside that interval:

```
W(f) = I0(pi alpha sqrt(1 - (2 f/F)^2)) / I0(pi alpha)
W(0) = 1;   q = F t;   U(q) = uW(q/F)
```

I0 is the modified Bessel function of order zero. Since W(0) = 1, the total area of w(t) is one. Its even symmetry gives U(-q) = 1 - U(q), so the 50% crossing is at t = 0.

Formula correction. With the NumPy convention sinc(x) = sin(pi x)/(pi x), the consistent analytic pair is w(t) = F sinc(sqrt((F t)^2 - alpha^2)) / I0(pi alpha), using its real hyperbolic continuation inside |F t| < alpha. The printed assignment has (pi alpha)^2 under this square root; that expression does not match its stated W(f). The implementation verifies the corrected pair numerically and computes U directly from the frequency window to avoid this ambiguity.

Part I | Kaiser input steps and rise time



Figures 1-2. Top: integrated unit-area Kaiser kernels for alpha = 0, 1, 2, 3, 4. Bottom: normalized central 10%-90% rise time, with computed crossing values annotated.

The normalized step is evaluated without truncating its negative-time tail:

```
U(q) = 1/2 + integral[0,1] W(F x/2) sin(pi q x)/(pi x) dx
F t10-90 = q90 - q10 = 2 q90, where U(q90) = 0.9
```

Larger alpha suppresses the high-frequency edge of W and reduces ringing, while widening the time-domain transition. At F = 40 GHz, the five rise times are 22.2910, 35.5189, 50.4830, 62.0853 and 71.8423 ps, respectively. The full numerical table is saved in results/part1_rise_times.csv.

Solving F t10-90 = 40 GHz x 40 ps = 1.6 gives alpha = 1.269110238; rounding alpha to 1.25 instead gives about 39.6934 ps. The curves use the corrected transform pair, and their rise times are solved from crossings rather than estimated from image pixels.

Part II | Circuit and reconstruction

2. Frequency response and reference planes

The assigned circuit is a 50 ohm line followed by an L-C-L ladder: L1 = L3 = 4.314 nH and C2 = 0.5301 pF, terminated in 50 ohm. It is the stated third-order Chebyshev low-pass example with nominal 5 GHz cutoff and 2 dB ripple. The Touchstone input is read using its declared units, complex representation, and two-port ordering S11, S21, S12, S22.

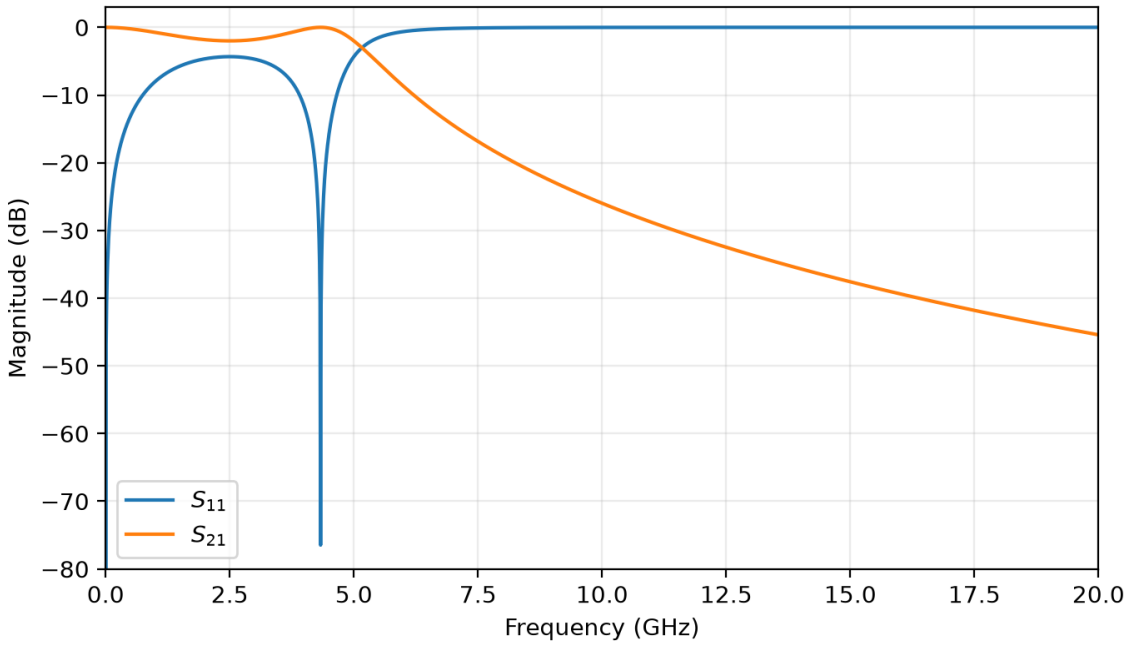


Figure 3. Supplied S11 and S21 magnitudes. S11 approaches a high-pass reflection response, making a sharp finite-band cutoff particularly visible in time.

Delay convention. The line parameter TD = 250 ps is one way. An input reflection traverses the line twice, so its physical propagation delay is 500 ps. This follows the circuit and S11 phase; the assignment prose calling 250 ps the reflected-wave delay is ambiguous. A separate 25 ps shift aligns the Kaiser 50% point with the center of the SPICE source ramp.

3. Frequency samples to a real step

The available samples run from DC to 20 GHz with spacing 10 MHz, so F = 2 fmax = 40 GHz and the periodic reconstruction interval is 1/df = 100 ns. The 524,288-point transform gives dt = 0.190735 ps. Zero padding refines time sampling; it does not add bandwidth or extend the period.

1. Form S11(f) W(f) exp(-j 2 pi f x 25 ps).

2. Retain DC once and create the negative-frequency conjugate samples. Apply half weight at the measured band edge for trapezoidal frequency integration.

3. Evaluate IFFT x N x df, center the periodic time axis, and retain the real impulse response.

4. Integrate with the cumulative trapezoidal rule from the negative-time boundary. Compare all methods on the same 0.5 ps grid over 0-3 ns.

This construction preserves the DC area, avoids duplicating DC, and uses the actual FFT time spacing instead of an unrelated display grid.

Part II | Independent circuit comparison

4. Match the excitation and extract the reflected wave

The standalone ngspice netlist uses a 2 V source, a 50 ohm source resistor, and a 50 ohm load. The launched incident wave is therefore vin/2, or 1 V after the edge. A 50 ps linear 0%-100% source ramp has a 40 ps 10%-90% rise. The reflected voltage at the source-side input reference plane is:

```
sNg(t) = v(input_port, t) - vin(t)/2
```

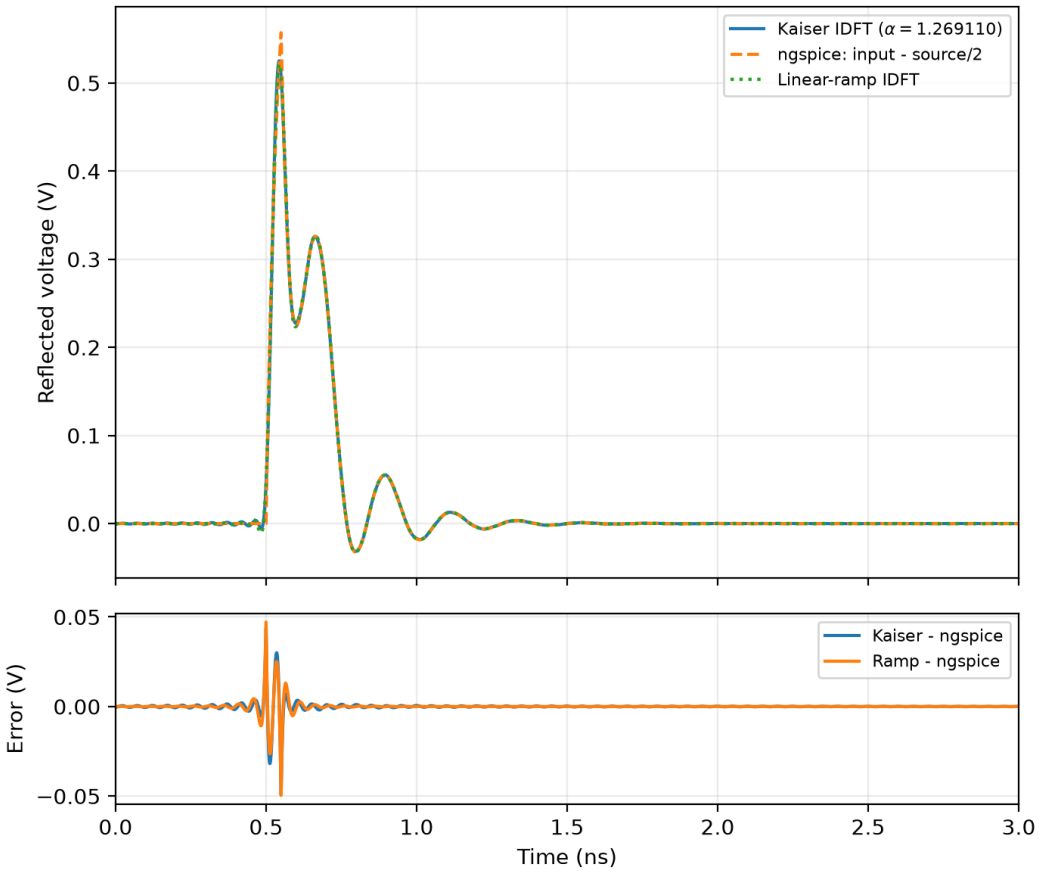


Figure 4. Reflected-wave comparison at the same reference plane. The lower panel shows residuals; the extra linear-ramp IDFT separates input-shape mismatch from reconstruction error.

Comparison on 0-3 ns / 6,001 samples	RMS error (mV)	Max absolute error (mV)
Kaiser IDFT vs ngspice	3.5362	49.2544
Linear-ramp IDFT vs ngspice	3.3966	49.5646
State space vs ngspice	0.0003	0.0197
Same-Kaiser state space vs IDFT	0.0004	0.0047

Equal rise time does not imply equal wave shape. The Kaiser input has smooth tails and a different derivative from the piecewise-linear SPICE ramp. Both reach 10%-90% in 40 ps, but their reflected peaks need not coincide exactly. For a closer excitation match, the diagnostic linear-ramp IDFT substitutes sinc(f x 50 ps) for W(f), keeping the same 25 ps center delay. Remaining error includes the 20 GHz bandwidth cutoff, sampled-data precision, and numerical integration. The 25 ps alignment is specified from source timing rather than tuned to minimize the reported residual.

Part II | Window tradeoff and verification

5. Repeat with alpha = 0 and alpha = 4

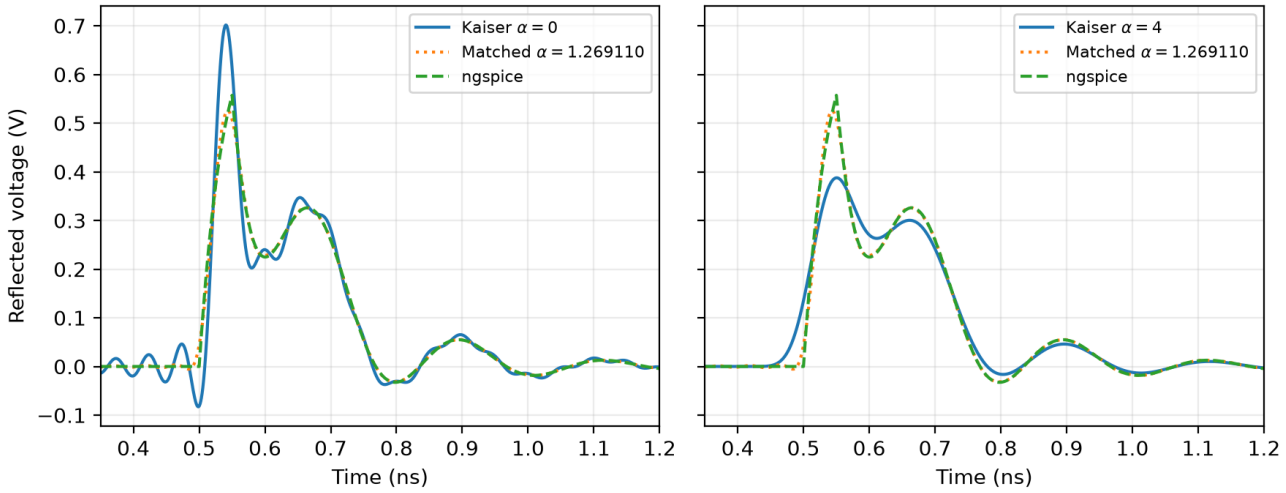


Figure 5. Alternative Kaiser parameters compared with the same independent circuit response. Alpha = 0 retains the abrupt spectral edge; alpha = 4 smooths the edge more strongly.

At alpha = 0, the rectangular frequency window retains the fastest central transition but creates visible pre-response and ringing around the reflected edge. At alpha = 4, stronger tapering reduces truncation oscillation and spreads the edge in time. Their actual rises are 22.2910 and 71.8423 ps at this bandwidth, so neither satisfies the requested 40 ps rise. The matched alpha provides the requested intermediate edge speed.

6. Independent evidence and limits

The circuit frequency response is independently computed with ABCD matrices. The supplied S11 agrees with the stated L-C-L circuit and line to about 8.73e-7 in maximum complex absolute error. Its use for reflection reconstruction is consistent with the assigned circuit.

The supplied S21 contains isolated 180-degree phase-sign anomalies at 2, 6, 10, 14 and 18 GHz. Its magnitude plot remains valid, and these samples do not affect this S11-only reflection calculation. The raw Touchstone data are retained unchanged; agreement of all complex S-parameters is not claimed.

A separate state-space model uses physical inductor currents and capacitor voltage and agrees with the linear-ramp ngspice run within 0.0197 mV. Driving that differential circuit with the same analytic Kaiser input, independently of S-parameters and FFT, matches the IDFT within 4.705 microvolts (RMS 0.371 microvolts). This supports the transform normalization and explains why the larger SPICE residual should not be interpreted as a reconstruction failure.

Doubling the FFT size changes the Kaiser response by at most 0.0250 mV over 0-3 ns. This tests time-sampling convergence; the finite bandwidth and input-shape differences remain physical limits of the linear-ramp comparison.

Reproducibility and submitted source

7. Rebuild the analysis and this report

From the homework/hw02 directory, install the declared Python dependencies and run:

```
python -m pip install -r report/requirements.txt
python src/hw2.py --ngspice /path/to/ngspice
python -m unittest discover -s tests -v
python report/build_report.py
```

Omit --ngspice to use an installed ngspice found on PATH, or the saved independent circuit reference if ngspice is unavailable. Recomputing a fresh reference is recommended when checking the circuit on a new platform. The program records whether the reference was recomputed or reused in results/verification.json. The archived simulation uses a fixed 2 V / 50 ps source rather than a waveform exported from the reconstructed response.

File or directory	Purpose
src/hw2.py	Complete rewritten analysis and figure generation
data/chebyshev_lpf.s2p	Supplied raw frequency-domain circuit data
simulation/chebyshev_reflection.cir	Independent transient circuit definition
simulation/spice_reference.csv	Saved circuit waveform for portable reruns
results/verification.json	Numeric metrics, FFT diagnostics and assertions
results/part1_rise_times.csv	Tabulated Kaiser rise times
results/part2_reflection.csv	Aligned reconstructed and reference waveforms
results/figures/	New raster and vector plots
report/report.md	Readable source of the main report
report/build_report.py	Portable report generator and source appendix

Saved verification outcome: 9/9 assertions pass. The checks cover window symmetry, the target rise time, real inverse transformation, DC area, FFT convergence, circuit agreement, and other explicit checks listed in verification.json. Unit tests provide additional checks of the rewritten computation; their exact test count is recorded by the test runner rather than inferred from this report.

8. Interpretation

Frequency-window multiplication specifies the finite-rise excitation used in a step reconstruction. At F = 40 GHz, alpha = 1.269110238 meets the requested 40 ps rise. A stronger taper trades edge speed for reduced truncation ringing. Independent circuit and state-space calculations validate the reflection mechanism and reference-plane timing, while the reported residuals expose the difference between a Kaiser step and a linear source ramp.

Reference material

The scope and circuit follow the supplied HW2 assignment, "Time-frequency transform (2) - Obtain step response using Kaiser window in IFT", especially its Parts I and II and source-code requirement. The accompanying course handout "Time-freq transform" supplies the LTI convolution and frequency/time-domain framework. Relative files in instructions/ preserve the provided homework reference material; the executable analysis above is newly written.

The following appendix includes the **complete analysis source and standalone SPICE netlist** in legible text. Line numbers refer to the accompanying editable files.

Appendix A | Complete Python analysis

Complete source: src/hw2.py. Line numbers mark original source lines; continued display lines are indented.

```
"""Reproducible NCU EE8091 HW2: Kaiser-window step responses and TDR.

Run from HW2 with ``python src/hw2.py``. To recompute the independent
ngspice reference, add ``--ngspice /path/to/ngspice``. All paths default
to this homework directory and all generated outputs are anonymous.

Fourier convention: h(t) = integral H(f) exp(+j*2*pi*f*t) df.
The assignment's F is twice the one-sided cutoff: F = 2*f_max.
"""

from __future__ import annotations

import argparse
import json
from dataclasses import dataclass
from pathlib import Path
import re
import shutil
import subprocess
import tempfile

import numpy as np
from numpy.polynomial.legendre import leggauss
from scipy.integrate import cumulative_trapezoid
from scipy.optimize import brentq
from scipy.signal import StateSpace, lsim


ROOT = Path(__file__).resolve().parents[1]
Z0 = 50.0
L = 4.314e-9
C = 0.5301e-12
ONE_WAY_DELAY = 250e-12
SOURCE_RAMP = 50e-12
TARGET_RISE = 40e-12
ALPHAS = np.arange(5, dtype=float)


@dataclass(frozen=True)
class Touchstone:
    """Touchstone 1.x S-parameter data, shaped [frequency, row, column]."""

    frequency_hz: np.ndarray
    s: np.ndarray
    reference_ohm: float


def read_touchstone(path: Path | str) -> Touchstone:
    """Read one- or two-port Touchstone 1.x RI, MA, or DB data.

    Honors frequency units, ignores comment tails, and permits wrapped
    numeric records. For two ports the standard order is S11,S21,S12,S22.
    Noise data and Touchstone 2.x keywords are deliberately rejected.
    """
    path = Path(path)
    match = re.fullmatch(r"\.s(\d+)p", path.suffix.lower())
    if match is None:
```

```
        raise ValueError("Expected a .sNp Touchstone filename")
    ports = int(match.group(1))
    if ports not in {1, 2}:
        raise ValueError("This homework reader supports one- and two-port files")
    # Touchstone defaults when an option line is absent.
    units, parameter, representation, impedance = "GHZ", "S", "MA", 50.0
    numbers: list[float] = []
    for line in path.read_text(encoding="utf-8-sig").splitlines():
        content = line.partition("!")[0].strip()
        if not content:
            continue
        if content.startswith("["):
            raise ValueError("This reader supports Touchstone 1.x, not 2.x")
        if content.startswith("#"):
            options = content[1:].upper().split()
            if len(options) < 3:
                raise ValueError("Incomplete Touchstone option line")
            units, parameter, representation = options[:3]
            if "R" in options:
                impedance = float(options[options.index("R") + 1])
            continue
        numbers.extend(float(x.replace("D", "E").replace("d", "e")) for x in
content.split())
    scales = {"HZ": 1.0, "KHZ": 1e3, "MHZ": 1e6, "GHZ": 1e9}
    if units not in scales or parameter != "S" or representation not in {"RI", "MA",
"DB"}:
        raise ValueError("Unsupported Touchstone units, parameter type, or
representation")
    width = 1 + 2 * ports * ports
    if not numbers or len(numbers) % width:
        raise ValueError("Incomplete full-matrix Touchstone data record")
    raw = np.asarray(numbers).reshape(-1, width)
    frequency = raw[:, 0] * scales[units]
    if np.any(np.diff(frequency) <= 0) or np.any(frequency < 0):
        raise ValueError("Frequencies must be nonnegative and strictly increasing")
    pairs = raw[:, 1:].reshape(-1, ports * ports, 2)
    first, second = pairs[..., 0], pairs[..., 1]
    if representation == "RI":
        values = first + 1j * second
    else:
        magnitude = 10 ** (first / 20) if representation == "DB" else first
        values = magnitude * np.exp(1j * np.deg2rad(second))
    # Column-major flattening of a matrix: S11,S21,S12,S22 for N=2.
    matrices = values.reshape(-1, ports, ports).transpose(0, 2, 1)
    return Touchstone(frequency, matrices, impedance)


def kaiser_spectrum(frequency_hz: np.ndarray, F: float, alpha: float) -> np.ndarray:
    """Continuous frequency window, equal to zero outside |f| <= F/2."""
    if F <= 0 or alpha < 0:
        raise ValueError("F must be positive and alpha nonnegative")
    x = 2 * np.asarray(frequency_hz, dtype=float) / F
    inside = np.abs(x) <= 1
    result = np.zeros_like(x)
    result[inside] = np.i0(np.pi * alpha * np.sqrt(np.maximum(0, 1 - x[inside] ** 2)))
    return result / np.i0(np.pi * alpha)


def kaiser_impulse(time_s: np.ndarray, F: float, alpha: float) -> np.ndarray:
    """Analytic inverse Fourier transform of the continuous Kaiser window.

    np.sinc(x)=sin(pi*x)/(pi*x); therefore the square root contains
```

```
    alpha**2, not (pi*alpha)**2. The hyperbolic branch is evaluated
    explicitly so the result is real without complex cancellation.
    """
    q = F * np.asarray(time_s, dtype=float)
    square = q * q - alpha * alpha
    out = np.ones_like(q)
    positive, negative = square > 1e-14, square < -1e-14
    out[positive] = np.sinc(np.sqrt(square[positive]))
    u = np.pi * np.sqrt(-square[negative])
    out[negative] = np.sinh(u) / u
    return F * out / np.i0(np.pi * alpha)


_GL_X, _GL_W = leggauss(256)
_QUAD_X = (_GL_X + 1) / 2
_QUAD_W = _GL_W / 2


def kaiser_step(q: np.ndarray | float, alpha: float) -> np.ndarray:
    """Zero-centered normalized Kaiser step U(q), q=F*t.

    U(q)=1/2 + integral_0^1 W(x)*sin(pi*q*x)/(pi*x) dx.
    This form includes the infinite past analytically, avoiding the
    baseline error from integrating only a short time interval.
    """
    q = np.asarray(q, dtype=float)
    flat = q.reshape(-1)
    window = np.i0(np.pi * alpha * np.sqrt(1 - _QUAD_X ** 2)) / np.i0(np.pi * alpha)
    result = np.empty_like(flat)
    for begin in range(0, flat.size, 4096):
        value = flat[begin:begin + 4096, None]
        integrand = value * np.sinc(value * _QUAD_X) * window
        result[begin:begin + 4096] = 0.5 + integrand @ _QUAD_W
    return result.reshape(q.shape)


def normalized_rise(alpha: float) -> float:
    """Return q90-q10 for the central 10%-90% transition."""
    positive_crossing = brentq(lambda q: float(kaiser_step(q, alpha)) - 0.9, 0, 20,
xtol=1e-13)
    return 2 * positive_crossing  # U(-q)=1-U(q).


def alpha_for_rise(F: float, rise_s: float) -> float:
    target = F * rise_s
    if target < normalized_rise(0):
        raise ValueError("Requested rise is faster than the alpha=0 window")
    return brentq(lambda a: normalized_rise(a) - target, 0, 20, xtol=1e-12)


def sparameters_from_circuit(frequency_hz: np.ndarray) -> np.ndarray:
    """Independent ABCD solution of the assigned line + L-C-L ladder."""
    frequency_hz = np.asarray(frequency_hz)
    omega = 2 * np.pi * frequency_hz
    n = frequency_hz.size
    series = np.zeros((n, 2, 2), complex)
    series[:, 0, 0] = series[:, 1, 1] = 1
    series[:, 0, 1] = 1j * omega * L
    shunt = np.zeros_like(series)
    shunt[:, 0, 0] = shunt[:, 1, 1] = 1
    shunt[:, 1, 0] = 1j * omega * C
    ladder = series @ shunt @ series
```

```
    A, B = ladder[:, 0, 0], ladder[:, 0, 1]
    cc, D = ladder[:, 1, 0], ladder[:, 1, 1]
    denominator = A + B / Z0 + cc * Z0 + D
    s11 = (A + B / Z0 - cc * Z0 - D) / denominator
    s21 = 2 / denominator
    result = np.zeros_like(ladder)
    result[:, 0, 0] = s11 * np.exp(-2j * omega * ONE_WAY_DELAY)
    result[:, 1, 0] = result[:, 0, 1] = s21 * np.exp(-1j * omega * ONE_WAY_DELAY)
    result[:, 1, 1] = s11
    return result


def windowed_reflection(
    frequency_hz: np.ndarray,
    s11: np.ndarray,
    alpha: float,
    delay_s: float = SOURCE_RAMP / 2,
    n_fft: int = 2 ** 19,
    source: str = "kaiser",
) -> tuple[np.ndarray, np.ndarray, np.ndarray, dict]:
    """Reconstruct continuous h and its step integral on an exact FFT grid.

    DC appears once; each nonzero positive-frequency sample has one
    conjugate negative-frequency partner. The last measured endpoint is
    half-weighted for the frequency-domain trapezoidal rule. Zero padding
    refines dt but does not extend bandwidth or the period 1/df.
    ``source='linear_ramp'`` substitutes sinc(f*T) for the Kaiser window.
    """
    frequency_hz = np.asarray(frequency_hz, dtype=float)
    s11 = np.asarray(s11, dtype=complex)
    df = frequency_hz[1] - frequency_hz[0]
    if frequency_hz[0] != 0 or not np.allclose(np.diff(frequency_hz), df, rtol=1e-10,
atol=1e-4):
        raise ValueError("IFFT requires a uniform frequency grid starting at DC")
    if n_fft % 2 or n_fft <= 2 * (frequency_hz.size - 1):
        raise ValueError("Use an even FFT size greater than twice the highest measured
index")
    if abs(s11[0].imag) > 1e-12:
        raise ValueError("Real impulse responses require a real DC point")
    F = 2 * frequency_hz[-1]
    if source == "kaiser":
        window = kaiser_spectrum(frequency_hz, F, alpha)
    elif source == "linear_ramp":
        window = np.sinc(frequency_hz * SOURCE_RAMP)
    else:
        raise ValueError("Unknown source shape")
    positive = s11 * window * np.exp(-2j * np.pi * frequency_hz * delay_s)
    positive[-1] *= 0.5
    full = np.zeros(n_fft, dtype=complex)
    full[:positive.size] = positive
    full[-positive.size + 1:] = positive[1:][::-1].conj()
    raw = np.fft.ifft(full) * n_fft * df
    impulse = np.fft.fftshift(raw.real)
    dt = 1 / (n_fft * df)
    time = (np.arange(n_fft) - n_fft // 2) * dt
    step = cumulative_trapezoid(impulse, dx=dt, initial=0)
    diagnostic = {
        "df_hz": float(df), "fmax_hz": float(frequency_hz[-1]),
        "F_hz": float(F), "n_fft": n_fft, "dt_s": float(dt),
        "period_s": float(1 / df),
        "dc_s11_real": float(s11[0].real),
        "max_ifft_imaginary": float(np.max(np.abs(raw.imag))),
```

```
        "max_ifft_imaginary_relative": float(np.max(np.abs(raw.imag)) / max(np.max(np.abs(raw.real)), 1)),
        "integrated_impulse": float(np.sum(impulse) * dt),
        "initial_step": float(step[0]), "final_step": float(step[-1]),
    }
    return time, impulse, step, diagnostic


def circuit_ramp_reflection(time_s: np.ndarray) -> np.ndarray:
    """Independent time-domain state-space reference, in volts.

    At the ladder input the source is 2*a in series with 50 ohm.
    States are [i_L1, v_C, i_L3]; reflected voltage is a-50*i_L1.
    Moving the reference plane through the matched line delays it by 2*TD.
    """
    time_s = np.asarray(time_s)
    t = time_s - time_s[0]
    incident = np.clip((time_s - 2 * ONE_WAY_DELAY) / SOURCE_RAMP, 0, 1)
    A = np.array([[-Z0 / L, -1 / L, 0], [1 / C, 0, -1 / C], [0, 1 / L, -Z0 / L]])
    B = np.array([[2 / L], [0], [0]])
    system = StateSpace(A, B, np.array([[-Z0, 0, 0]]), np.array([[1.0]]))
    _, reflected, _ = lsim(system, U=incident, T=t, interp=True)
    return reflected


def circuit_kaiser_reflection(time_s: np.ndarray, F: float, alpha: float) -> np.ndarray:
    """Independent differential-circuit response to the same Kaiser edge.

    Integrate the analytic impulse on a 0.2 ps grid with 20 ns prehistory;
    center the incident edge at U(0)=0.5. This calculation uses neither the
    supplied S-parameters nor an FFT. Voltage-scaled inductor states and
    nanosecond units keep the three circuit equations well scaled.
    """
    time_s = np.asarray(time_s)
    step_ns = 0.0002
    end_ns = max(4.0, float(np.max(time_s)) * 1e9 + 1)
    samples = int(np.ceil((end_ns + 20) / step_ns))
    dense_ns = -20 + np.arange(samples + 1) * step_ns
    impulse_per_ns = kaiser_impulse(dense_ns * 1e-9, F, alpha) * 1e-9
    incident = cumulative_trapezoid(impulse_per_ns, dx=step_ns, initial=0)
    incident += .5 - np.interp(0, dense_ns, incident)
    k = Z0 / L * 1e-9
    k2 = 1 / (Z0 * C) * 1e-9
    A = np.array([[-k, -k, 0], [k2, 0, -k2], [0, k, -k]])
    B = np.array([[2 * k], [0], [0]])
    system = StateSpace(A, B, np.array([[-1, 0, 0]]), np.array([[1.0]]))
    _, reflected, _ = lsim(system, U=incident, T=dense_ns - dense_ns[0], interp=True)
    delay_ns = (2 * ONE_WAY_DELAY + SOURCE_RAMP / 2) * 1e9
    return np.interp(time_s * 1e9 - delay_ns, dense_ns, reflected)


def run_ngspice(executable: str, simulation: Path, scratch_parent: Path | None = None) -> np.ndarray:
    """Run the saved anonymous standalone circuit, returning time and volts."""
    with tempfile.TemporaryDirectory(prefix="hw02-ngspice-", dir=scratch_parent or simulation) as directory:
        scratch = Path(directory)
        shutil.copy2(simulation / "chebyshev_reflection.cir", scratch)
        completed = subprocess.run(
            [executable, "-b", "chebyshev_reflection.cir"], cwd=scratch,
            stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True,
            encoding="utf-8", errors="replace", timeout=120,
```

```
        )
        # Output can include machine-specific install paths. Do not persist it.
        result = scratch / "spice_reference.txt"
        if completed.returncode or not result.exists():
            raise RuntimeError("ngspice failed; inspect the standalone circuit and your local installation")
        values = np.loadtxt(result)
    if values.ndim != 2 or values.shape[1] < 4:
        raise RuntimeError("Unexpected ngspice wrdata columns")
    # First column is time; the saved vectors are source, input port, reflection.
    return values[:, [0, 1, 2, 3]]


def error_metrics(observed: np.ndarray, reference: np.ndarray) -> dict:
    residual = np.asarray(observed) - np.asarray(reference)
    return {"rmse_v": float(np.sqrt(np.mean(residual ** 2))),
            "max_abs_error_v": float(np.max(np.abs(residual)))}


def save_csv(path: Path, columns: list[np.ndarray], names: list[str]) -> None:
    np.savetxt(path, np.column_stack(columns), delimiter=",", header=",".join(names), comments="", fmt="%.12e")


def make_figures(
    results: Path, data: Touchstone, rows: list[dict],
    time: np.ndarray, kaiser_response: np.ndarray, ramp_response: np.ndarray,
    spice: np.ndarray, state: np.ndarray, alpha: float,
    alpha_responses: dict[float, np.ndarray],
) -> None:
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": 0.25,
                         "figure.dpi": 150, "savefig.dpi": 200})
    destination = results / "figures"
    destination.mkdir(exist_ok=True)
    def save(figure, name):
        figure.tight_layout()
        figure.savefig(destination / (name + ".png"))
        figure.savefig(destination / (name + ".pdf"), metadata={"Author": "", "Creator": "", "Title": name})
        plt.close(figure)
    q = np.linspace(-5, 5, 4001)
    fig, (left, right) = plt.subplots(1, 2, figsize=(8.5, 3.6))
    x = np.linspace(-1.15, 1.15, 2001)
    for a in ALPHAS:
        left.plot(x, kaiser_spectrum(x / 2, 1, a), label=rf"$\alpha={a:.0f}$")
        right.plot(q, kaiser_impulse(q, 1, a))
    left.set(xlabel=r"Normalized frequency $2f/F$", ylabel=r"$W(f)$", xlim=(-1.15, 1.15))
    left.legend(fontsize=8)
    right.set(xlabel=r"Normalized time $Ft$", ylabel=r"Normalized impulse $w(t)/F$", xlim=(-5, 5))
    save(fig, "part1_kaiser_window_impulse")
    fig, ax = plt.subplots(figsize=(7.0, 4.1))
    for a in ALPHAS:
        ax.plot(q, kaiser_step(q, a), label=rf"$\alpha={a:.0f}$")
    ax.axhline(.1, color="0.65", lw=.7, ls=":")
    ax.axhline(.9, color="0.65", lw=.7, ls=":")
    ax.set(xlabel=r"Normalized time $Ft$", ylabel=r"Kaiser step $u_W(t)$", ylim=(-.15, 1.15), xlim=(-5, 5))
```

```
    ax.legend(ncol=3, fontsize=9)
    save(fig, "part1_kaiser_steps")
    fig, ax = plt.subplots(figsize=(6.5, 3.8))
    ax.plot([r["alpha"] for r in rows], [r["normalized_rise"] for r in rows], "o-")
    ax.set(xlabel=r"Kaiser parameter $\alpha$", ylabel=r"Normalized rise time $Ft_{10-90}$", xticks=ALPHAS)
    save(fig, "part1_rise_time")
    fig, ax = plt.subplots(figsize=(7.0, 4.1))
    for row, col, label in [(0, 0, "$S_{11}$"), (1, 0, "$S_{21}$")]:
        magnitude = np.abs(data.s[:, row, col])
        ax.plot(data.frequency_hz / 1e9, 20 * np.log10(np.maximum(magnitude, 1e-12)), label=label)
    ax.set(xlabel="Frequency (GHz)", ylabel="Magnitude (dB)", xlim=(0, 20), ylim=(-80, 3))
    ax.legend()
    save(fig, "part2_sparameters")
    fig, (ax, bottom) = plt.subplots(2, 1, figsize=(7.1, 6), sharex=True, gridspec_kw={"height_ratios": [3, 1]})
    ax.plot(time * 1e9, kaiser_response, label=rf"Kaiser IDFT ($\alpha={alpha:.6f}$)")
    ax.plot(time * 1e9, spice, "--", label="ngspice: input - source/2")
    ax.plot(time * 1e9, ramp_response, ":", label="Linear-ramp IDFT", lw=1.6)
    ax.set(ylabel="Reflected voltage (V)", xlim=(0, 3))
    ax.legend(fontsize=8)
    bottom.plot(time * 1e9, kaiser_response - spice, label="Kaiser - ngspice")
    bottom.plot(time * 1e9, ramp_response - spice, label="Ramp - ngspice")
    bottom.set(xlabel="Time (ns)", ylabel="Error (V)")
    bottom.legend(fontsize=8)
    save(fig, "part2_reflection_comparison")
    fig, ax = plt.subplots(figsize=(6.5, 4))
    ax.plot(time * 1e9, kaiser_response, label="Kaiser IDFT")
    ax.plot(time * 1e9, spice, "--", label="ngspice")
    ax.set(xlabel="Time (ns)", ylabel="Reflected voltage (V)", xlim=(.45, .75))
    ax.legend()
    save(fig, "part2_reflection_edge")
    fig, axes = plt.subplots(1, 2, figsize=(9, 3.6), sharey=True)
    for a, ax in zip([0.0, 4.0], axes):
        ax.plot(time * 1e9, alpha_responses[a], label=rf"Kaiser $\alpha={a:.0f}$")
        ax.plot(time * 1e9, kaiser_response, ":", label=rf"Matched $\alpha={alpha:.6f}$")
        ax.plot(time * 1e9, spice, "--", label="ngspice")
        ax.set(xlabel="Time (ns)", xlim=(0.35, 1.2))
        ax.legend(fontsize=8)
    axes[0].set_ylabel("Reflected voltage (V)")
    save(fig, "part2_alpha_comparison")


def main(argv: list[str] | None = None) -> None:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--ngspice", help="Executable for a fresh independent circuit simulation")
    parser.add_argument("--output", type=Path, help="Alternative output directory; default is HW2/results")
    parser.add_argument("--n-fft", type=int, default=2 ** 19)
    args = parser.parse_args(argv)
    results = args.output.resolve() if args.output else ROOT / "results"
    simulation = ROOT / "simulation"
    results.mkdir(parents=True, exist_ok=True)
    data = read_touchstone(ROOT / "data" / "chebyshev_lpf.s2p")
    frequency = data.frequency_hz
    F = 2 * frequency[-1]
    alpha = alpha_for_rise(F, TARGET_RISE)
    rows = [{"alpha": float(a), "normalized_rise": normalized_rise(a),
```

```
408             "rise_ps_at_F_40GHz": normalized_rise(a) / F * 1e12} for a in ALPHAS]
409     save_csv(results / "part1_rise_times.csv",
410             [np.array([row[key] for row in rows]) for key in rows[0]], list(rows[0]))
411     q = np.linspace(-5, 5, 4001)
412     save_csv(results / "part1_steps.csv", [q] + [kaiser_step(q, a) for a in ALPHAS],
413             ["normalized_time_Ft"] + [f"step_alpha_{int(a)}" for a in ALPHAS])
414     t_fft, impulse, step, grid = windowed_reflection(frequency, data.s[:, 0, 0], alpha,
    n_fft=args.n_fft)
415     _, _, ramp_step, _ = windowed_reflection(frequency, data.s[:, 0, 0], alpha,
    n_fft=args.n_fft, source="linear_ramp")
416     _, _, coarse_step, _ = windowed_reflection(frequency, data.s[:, 0, 0], alpha,
    n_fft=args.n_fft // 2)
417     t_coarse = (np.arange(args.n_fft // 2) - args.n_fft // 4) / (args.n_fft // 2 *
    grid["df_hz"])
418     # A common 0.5 ps grid makes reference comparisons and CSV reuse simple.
419     time = np.linspace(0, 3e-9, 6001)
420     kaiser_response = np.interp(time, t_fft, step)
421     ramp_response = np.interp(time, t_fft, ramp_step)
422     alpha_responses = {}
423     for a in (0.0, 4.0):
424         _, _, response, _ = windowed_reflection(frequency, data.s[:, 0, 0], a,
    n_fft=args.n_fft)
425         alpha_responses[a] = np.interp(time, t_fft, response)
426     state = circuit_ramp_reflection(time)
427     same_kaiser_state = circuit_kaiser_reflection(time, F, alpha)
428     executable = args.ngspice or shutil.which("ngspice") or shutil.which("ngspice_con")
429     if executable:
430         raw_spice = run_ngspice(executable, simulation, results)
431         reference_output = results / "spice_reference.csv" if args.output else
    simulation / "spice_reference.csv"
432         save_csv(reference_output, [raw_spice[:, i] for i in range(4)],
433                 ["time_s", "source_v", "input_port_v", "reflected_v"])
434         spice_status = "recomputed"
435     else:
436         reference = simulation / "spice_reference.csv"
437         if not reference.exists():
438             raise RuntimeError("ngspice is unavailable and no saved independent
    reference exists")
439         raw_spice = np.loadtxt(reference, delimiter=",", skiprows=1)
440         spice_status = "saved_reference"
441     spice = np.interp(time, raw_spice[:, 0], raw_spice[:, 3])
442     save_csv(results / "part2_reflection.csv", [time, kaiser_response, ramp_response,
    spice, state, same_kaiser_state],
443             ["time_s", "kaiser_idft_v", "linear_ramp_idft_v", "ngspice_reflected_v",
    "state_space_reflected_v", "same_kaiser_state_space_v"])
444     modeled = sparameters_from_circuit(frequency)
445     s11_error = float(np.max(np.abs(modeled[:, 0, 0] - data.s[:, 0, 0])))
446     s21_error = np.abs(modeled[:, 1, 0] - data.s[:, 1, 0])
447     anomaly = s21_error > 1e-4
448     coarse = np.interp(time, t_coarse, coarse_step)
449     convergence = error_metrics(kaiser_response, coarse)
450     checks = {
451         "kaiser_impulse_even": bool(np.allclose(kaiser_impulse(q, 1, alpha),
    kaiser_impulse(-q, 1, alpha), atol=1e-13)),
452         "kaiser_step_symmetry": bool(np.allclose(kaiser_step(q, alpha) + kaiser_step(-q,
     alpha), 1, atol=1e-12)),
453         "target_rise_within_0_001ps": bool(abs(normalized_rise(alpha) / F - TARGET_RISE)
     < 1e-15),
454         "real_ifft": bool(grid["max_ifft_imaginary_relative"] < 1e-14),
455         "dc_area_matches_s11": bool(abs(grid["integrated_impulse"] - data.s[0, 0,
    0].real) < 1e-12),
456         "fft_convergence_below_0_1mV": bool(convergence["max_abs_error_v"] < 1e-4),
```

```
457         "ngspice_vs_state_below_0_1mV": bool(error_metrics(spice,
    state)["max_abs_error_v"] < 1e-4),
458         "supplied_s11_matches_circuit_below_1ppm": bool(s11_error < 1e-6),
459         "same_kaiser_state_vs_idft_below_0_1mV": bool(error_metrics(kaiser_response,
    same_kaiser_state)["max_abs_error_v"] < 1e-4),
460     }
461     metrics = {
462         "course": "NCU EE8091", "homework": "HW2", "all_checks_passed":
    all(checks.values()),
463         "part1": rows,
464         "part2": {"alpha_for_40ps": float(alpha), "target_rise_ps": 40,
465                   "actual_kaiser_rise_ps": float(normalized_rise(alpha) / F * 1e12),
466                   "source_full_ramp_ps": SOURCE_RAMP * 1e12,
467                   "source_ramp_10_90_ps": SOURCE_RAMP * .8 * 1e12,
468                   "source_center_delay_ps": SOURCE_RAMP / 2 * 1e12,
469                   "line_one_way_delay_ps": ONE_WAY_DELAY * 1e12,
470                   "reflection_round_trip_delay_ps": 2 * ONE_WAY_DELAY * 1e12,
471                   "spice_reference_status": spice_status,
472                   "s11_circuit_max_complex_error": s11_error,
473                   "s21_phase_anomaly_frequencies_ghz": (frequency[anomaly] /
    1e9).tolist(),
474                   "s21_other_points_max_complex_error":
    float(np.max(s21_error[~anomaly])),
475                   "s21_max_magnitude_error": float(np.max(np.abs(np.abs(modeled[:, 1,
    0]) - np.abs(data.s[:, 1, 0])))),
476                   "kaiser_vs_ngspice": error_metrics(kaiser_response, spice),
477                   "linear_ramp_idft_vs_ngspice": error_metrics(ramp_response, spice),
478                   "state_space_vs_ngspice": error_metrics(state, spice),
479                   "same_kaiser_state_vs_idft": error_metrics(same_kaiser_state,
    kaiser_response),
480                   "same_kaiser_state_grid": {"dt_ps": .2, "prehistory_ns": 20,
    "uses_sparameters_or_fft": False},
481                   "fft_size_doubled_convergence": convergence,
482                   "alpha_comparison": [{"alpha": a, "rise_ps": normalized_rise(a) / F *
    1e12,
483                                         "peak_v": float(np.max(alpha_responses[a])),
484                                         "minimum_v": float(np.min(alpha_responses[a])),
485                                         "peak_time_ns":
    float(time[np.argmax(alpha_responses[a])] * 1e9),
486                                         **error_metrics(alpha_responses[a], spice)} for
    a in (0.0, 4.0)],
487                   "comparison_interval_ns": [0, 3], "comparison_samples": time.size,
488                   "grid": grid},
489         "checks": checks,
490         "notes": [
491             "Kaiser and piecewise-linear edges share the 40 ps 10-90 rise but differ in
    shape.",
492             "Finite 20 GHz bandwidth and the window cause residual disagreement with
    broadband SPICE.",
493             "The line TD is one way; input reflection accumulates twice that delay.",
494             "Saved SPICE reference is generated independently from the standalone
    netlist.",
495             "The supplied S11 agrees with the nominal circuit within export rounding.",
496             "Supplied S21 has isolated 180-degree phase anomalies at 2,6,10,14,18 GHz;
    preserved as source data and not used in reflection.",
497         ],
498     }
499     (results / "verification.json").write_text(json.dumps(metrics, indent=2) + "\n",
    encoding="utf-8")
500     save_csv(results / "part2_alpha_comparison.csv", [time, alpha_responses[0],
    kaiser_response, alpha_responses[4], spice],
501              ["time_s", "kaiser_alpha0_v", "kaiser_matched_v", "kaiser_alpha4_v",
```

```
    "ngspice_reflected_v"])
502     make_figures(results, data, rows, time, kaiser_response, ramp_response, spice,
    state, alpha, alpha_responses)
503     print(json.dumps({"all_checks_passed": metrics["all_checks_passed"], "alpha": alpha,
504                       "kaiser_vs_ngspice": metrics["part2"]["kaiser_vs_ngspice"],
505                       "output_directory": "results"}, indent=2))
506     if not metrics["all_checks_passed"]:
507         raise SystemExit("One or more numerical checks failed; see
    results/verification.json")
508
509
510 if __name__ == "__main__":
511     main()
```

Appendix B | Independent SPICE netlist

Complete source: simulation/chebyshev_reflection.cir. Line numbers mark original source lines; continued display lines are indented.

```
 1  * NCU EE8091 HW2: 3rd-order Chebyshev LPF, 2 dB ripple, 5 GHz cutoff.
 2  * Input reference plane is node 2. Source is 2 V with a 50 ps full ramp.
 3  * Incident wave is V(1)/2; reflected wave is V(2)-V(1)/2.
 4  * T1 TD=250 ps is one way, so its reflection round trip is 500 ps.
 5  Vin 1 0 PULSE(0 2 0 50p 50p 1u 10u)
 6  Rin 1 2 50
 7  T1 2 0 3 0 Z0=50 TD=250p
 8  L1 3 4 4.314n
 9  C2 4 0 0.5301p
10  L3 4 5 4.314n
11  Rout 5 0 50
12  .options reltol=1e-9 abstol=1e-15 vntol=1e-12 method=trap
13  .control
14  set noaskquit
15  set numdgt=15
16  tran 0.5p 3n 0 0.01p
17  linearize v(1) v(2)
18  let reflected = v(2)-v(1)/2
19  set wr_singlescale
20  wrdata spice_reference.txt v(1) v(2) reflected
21  quit
22  .endc
23  .end
```